

Omni-Compass against every major cluster platform: A, B and C

Referee report, 26 September 2026. Repository Omni-Compass/The-Omni-Compass-Control-Core-Engine, branch claude/kubernetes-clusters-docker-stack-gp26ve, commit a17c014. Every number is generated by `python tools/abc_report.py` from result files in the repository. Each result states how it was obtained: **measured** on a live Kubernetes control plane, **simulated** in this repository's plants, or **emulated** (a competitor reproduced from its public documentation, not its binary).

1. The answer

Three architectures were measured against seven platforms: Kubernetes, Red Hat OpenShift, Google GKE, Azure AKS (node auto-provisioning, which is Karpenter and also stands for Amazon EKS Auto Mode), IBM Turbonomic, CAST AI and Spot Ocean.

- **A. The platform alone.** Its own autoscalers decide.
- **B. Omni-Compass on top of the platform.** The platform's controllers keep running; the six-state engine governs on top.
- **C. Omni-Compass alone.** The engine is the single authority; the platform's managers do not decide.

B against A (held-out, 30 unseen scenarios per workload, settings frozen by SHA-256 first): 363 of 364 gauge-by-platform-by-workload cells are equal or better; 1 is worse: batch jobs on CAST AI, scale reversals 7.7% worse. Batch jobs gain most: the slowest jobs finish much sooner on every platform (section 5.3). Where no intervention was free (web services, four clusters) Omni-Compass reproduces the platform exactly.

C against the seven platforms at once (held-out): 63 cells where some platform is better (section 6). Part of that gap is not closable by any controller: a perfect controller that knows the future cannot have both the tightest packer's machine-hours and the laziest autoscaler's machine churn (section 6.2).

Live Kubernetes (measured, run 36221702999, every muscle on, B architecture): worker machines 6.00 to 3.19, energy -3.2% with parked machines on standby, response time p95 491 to 383 ms and p99 613 to 491 ms, 0 failed requests; one significant negative (energy per CPU core-hour, section 7).

2. The rule, and how a cell is judged

- A **cell** is one gauge, one competitor, one workload. There are 13 gauges: energy, machine-hours, response time (p95, p99, mean), time over the backlog, power and heat limits, machine starts+stops, scale reversals, pod changes, work completed and time healthy.
- Omni-Compass **loses** a cell when it is worse by more than 0.5% (0.5 events for counts) **and** the paired bootstrap 95% interval of the per-scenario difference excludes zero. Otherwise the cell is equal (identical, or not significantly different) or better.
- Settings are chosen on **development** scenarios only, frozen with SHA-256 hashes (`tuning/B_PREREGISTRATION*.json`, `results/muscles/PREREGISTRATION.json`), then run once on **held-out** scenarios never used for tuning.
- Scenario generator: `fleet/harness.py` (15-second resolution, 6 hours; boot delay, power model, thermal model, site power limit identical for every arm). Response time: `M/M/c` queueing per workload plus backlog drain (`fleet/sim_slo.py`).

3. The opponents, and how each is reproduced

Platform	What decides	Reproduced as (parameters)
Kubernetes	HPA + Cluster Autoscaler	HPA 70%; remove a node after 10 min below 50% request use, not within 10 min of an add
OpenShift	ClusterAutoscaler resource	documented example: threshold 0.4, unneeded 5 min, delay after add 10 min
GKE (optimize-utilization)	MostAllocated packing, aggressive scale-down	declared: threshold 0.65, 2 min (Google publishes no numbers)
AKS NAP (= Karpenter, EKS Auto Mode)	consolidation WhenEmptyOrUnderutilized	consolidateAfter 0 s; remove when the rest hold all requests at 90% packing
IBM Turbonomic	market-based resize + suspend	requests resized every 10 min to per-pod p99 usage; nodes suspended toward 70% packing
CAST AI	Evictor bin-packing	every 60 s, drain a node older than 5 min when its pods fit elsewhere
Spot Ocean	headroom autoscaler	5% automatic headroom kept; least-used node removed when pods fit elsewhere

These are emulations of documented behaviour on the same plant, not the vendors' software; each vendor's full product has more features than its autoscaling core. Every platform runs its pods at the same HPA target (70%), so pod behaviour is identical and differences come from how machines are managed.

4. What Omni-Compass is, mechanically

A six-state dynamical system, $x = (E, U, I_U, S, B, B_{\text{dot}})$: energy/excitation, order (a double-well with a healthy basin at $U = +1$), integrated unmet need, stress (a cubic potential), and a damped second-order bath. Equations (1)-(8) and every term's role are in `docs/MECHANISM_OF_ACTION.md`. Telemetry is blended into the state (weight 0.339) and the equations are integrated with RK4 each decision.

Mode (Jacobian eigenvalue at the operating point)	Time constant (decisions)	Period (decisions)
-7.90+0.00i	1.3	
-3.10+0.00i	3.2	
-0.72+0.00i	13.9	
-0.20+0.98i	50.0	64
-0.20-0.98i	50.0	64
-0.17+0.00i	59.8	

All modes are stable: the operating point ($U = 0.976$) sits inside the healthy basin. The equations and the observation blend each move the state about half of every step. Compute: 95 microseconds per decision in Python, about 3 in the C++ twin.

5. B: Omni-Compass on top of each platform, against the same platform alone (held-out)

Source tuning/B_LEAGUE_HELDOUT2.json. Omni-Compass keeps the platform running and intervenes only where the engine sees it will pay: it undoes a machine removal that is about to be reversed (while requests are rising or the engine has not converged), and adds early the machine the platform would add only after pods go pending. With both off it reproduces the platform exactly (checked on every platform and workload); every gain below is on top of that floor.

5.1 Web services

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	-	13 of 13
GKE (optimize)	-	-	-	13 of 13
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	-	13 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

5.2 Four clusters, one site

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	-	13 of 13
GKE (optimize)	-	-	-	13 of 13
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	-	13 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

5.3 Batch jobs

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 12, trend over 16), reversal veto; OpenShift: early add (lead 12, trend over 16), reversal veto; GKE (optimize): early add (lead 6, trend over 8), reversal veto; AKS NAP (Karpenter): early add (lead 3, trend over 16), reversal veto; Turbonomic: early add (lead 6, trend over 4), reversal veto; CAST AI: early add (lead 12, trend over 16), reversal veto; Spot Ocean: early add (lead 3, trend over 16), reversal veto.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	response p99 -91%; response mean -23%; machine starts+stops -16%; scale reversals -70%	-	-	9 of 13
OpenShift	response p99 -91%; response mean -25%; machine starts+stops -23%; scale reversals -87%	-	-	9 of 13
GKE (optimize)	response p99 -91%; response mean -27%; machine starts+stops -22%	-	-	10 of 13
AKS NAP (Karpenter)	response p95 -57%; response p99 -24%; response mean -19%; time over backlog limit -47%; machine starts+stops -13%	-	energy +0.2%; machine-hours +0.5%	6 of 13
Turbonomic	response p99 -95%; response mean -34%; machine starts+stops -16%	-	-	10 of 13
CAST AI	response p95 -47%; response p99 -25%; response mean -32%; time over backlog limit -100%; machine starts+stops -9%	scale reversals +7.7%	-	7 of 13
Spot Ocean	response p95 -12%; response p99 -9%; response mean -8%; time over backlog limit -100%; machine starts+stops -8%; scale reversals -8%	-	energy +0.1%; machine-hours +0.3%	5 of 13

5.4 Gpu training

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 3, trend over 8), reversal veto; OpenShift: early add (lead 3, trend over 16); GKE (optimize): pass-through; AKS NAP (Karpenter): pass-through; Turbonomic: early add (lead 3, trend over 8); CAST AI: pass-through; Spot Ocean: pass-through.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	time over heat limit +0.1%	12 of 13
GKE (optimize)	-	-	-	13 of 13
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	energy +0.1%; time over heat limit +0.2%; machine-hours +0.4%	10 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

Percentages are the plain change of Omni-Compass on top against the platform alone: for response times, starts+stops, reversals, energy and machine-hours negative is better; for work completed and time healthy positive is better. A LOSS is worse beyond the 0.5% tolerance with a 95% interval excluding zero (section 2).

6. C: Omni-Compass alone, against all seven platforms at once

Source tuning/LEAGUE_HELDOUT.json. One Omni-Compass configuration per workload type (chosen on development scenarios): web: speed159, multi: speed159, batch: speed112, gpu: omni_fleet.

Workload	Gauge	Platforms better than Omni-Compass	Omni worse by (range)
batch	energy	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	1.7% to 6.7%
batch	machine-hours	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	4.4% to 18.0%
batch	scale reversals	Kubernetes, OpenShift	266.7% to 279.3%
batch	machine starts+stops	Kubernetes, OpenShift	54.3% to 64.9%
gpu	machine-hours	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	5.5% to 14.4%
gpu	scale reversals	Kubernetes, OpenShift	85.6% to 87.6%
gpu	machine starts+stops	Kubernetes, OpenShift	37.6% to 49.8%
multi	energy	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	23.9% to 64.7%
multi	machine-hours	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	36.7% to 119.0%
multi	pod changes	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean	8.2% to 10.3%
web	energy	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	23.9% to 63.5%
web	machine-hours	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	36.8% to 116.3%
web	pod changes	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean	7.8% to 10.8%

6.1 Where C wins

- Web services: response p95 110.1 ms vs best platform 124.9 ms; response p99 122.8 ms vs best platform 546 ms; response mean 145.3 ms vs best platform 216.2 ms.
- Four clusters, one site: response p95 112.1 ms vs best platform 126.9 ms; response p99 137.8 ms vs best platform 262.9 ms; response mean 145.6 ms vs best platform 217.2 ms.
- Batch jobs: response p99 337.4 ms vs best platform 1277 ms; response mean 115.2 ms vs best platform 135.6 ms.

6.2 The physical limit: what no controller can do

For each scenario, dynamic programming with perfect knowledge of the future (boot delay ignored, which only helps it) finds the fewest machine starts+stops that can hold every pod at a given machine-hour budget (tuning/bound.py).

Workload	Scenario	Tightest platform machine-hours	Fewest starts+stops of any platform	Perfect controller at those machine-hours
web	101	18.4	9	11
web	102	18.3	9	9
web	103	31.4	4	26
web	104	22.0	12	24
multi	101	74.9	36	46
multi	102	112.7	26	100
multi	103	122.5	17	114
multi	104	93.4	35	91
batch	101	108.0	24	67

Workload	Scenario	Tightest platform machine-hours	Fewest starts+stops of any platform	Perfect controller at those machine-hours
batch	102	56.8	7	33
batch	103	66.9	12	44
batch	104	93.4	15	58
gpu	101	66.4	12	28
gpu	102	57.5	16	36
gpu	103	51.5	19	30
gpu	104	57.0	20	26

In 15 of 16 scenarios even a perfect controller needs more starts+stops than the laziest platform to reach the tightest platform's machine-hours. Those two platforms sit at opposite ends of one physical trade; beating both at once is impossible for any software. In C, a loss on one side of that pair is the price of a win on the other.

6.3 Letting the equations drive directly

A governor in which the equations run closed-loop and their own quantities (control effort, unmet need I_U , the bath's rate) drive every muscle (omnicompass/mathdrive.py) was tested on 96 settings per workload: 66 losing cells, the same trade-off as the rule-based wiring. The engine clock alone moved batch p99 from 3.9 s to 0.1 s at equal energy.

7. Live Kubernetes (measured)

Two identical kind clusters (1 control plane + 6 workers) at the same time, same load; one with Omni-Compass on top (B). Omni runs as a least-privilege service account whose permissions are proven with `kubectl auth can-i` receipts before each run.

Run	Change	Validity	p95 (ms)	p99 (ms)	Energy
36213152881	HPA target, node pool, power cap (in-place pod CPU limits), heat (harm)	valid	486 to 802	675 to 1,113	222 to 215
36214629046	latency afferent (p95 over 500 ms SLO as queue pressure) and power-cap	valid	481 to 600	596 to 777	222 to 219
36216647786	SLO reflex; eviction receipt fixed	INVALID as a test of the engine	479 to 320	583 to 420	224 to 220
36218637030	controller fail-safe; benchmark prints controller log and rejects earl	PARTIAL	491 to 492	645 to 683	223 to 221
36220059046	get on pods/resize (power cap can apply)	INVALID for response time and energy	501 to 380	660 to 415	224 to 220
36221702999	probe tunnel restarts on both arms; power cap able to apply	VALID	491 to 383	613 to 491	224 to 216

Run 6 in full:

Gauge	Kubernetes alone	+ Omni-Compass	Change
Worker nodes in service, mean	6.00	3.19	-46.8%

Gauge	Kubernetes alone	+ Omni-Compass	Change
Node-hours	2.02	1.07	-47.0%
Energy (Wh), standby counted	224	216	-3.2%
Power (W), peak	710	682	-3.8%
Energy per core-hour (Wh)	742	1,041	+40.2% worse (significant)
Utilisation	0.074	0.097	+30.2%
Pending pods, pod-minutes	0.283	0.533	+88% (n.s.)
HPA shortfall, minutes	0.817	1.33	+63% (n.s.)
Response time (ms), mean	264	165	-37.4%
Response time (ms), median	227	119	-47.4%
Response time (ms), 95th percentile	491	383	-22.1%
Response time (ms), 99th percentile	613	491	-19.9%
Failed requests (%)	0	0	0

First fully valid run with every live muscle on: fewer servers (-47%), less energy (-3.2%, standby counted), lower peak power, and faster responses at every percentile. The one significant negative is energy per core-hour (+48%): the power cap lowers the CPU the app consumes (-31%), and that CPU is the metric's denominator. Whether energy per request served also rose is not measured (the load generator's request count is not captured); a per-request energy gauge is added to the capture next.

8. The nine problem-map muscles (simulated, held-out, Omni-Compass direct against the strongest native tool)

Muscle	Opponent	Better	Worse
rightsize	hpa_vpa	cpu core hours -3%; mem gib hours -11%; p99 ms -99%; replica reversals -84%; slo breach min -57%	p95 ms +15%; oom kills +531%
coldstart	keda	p95 ms -75%; p99 ms -58%; delayed 1s pct -71%; instance hours -8%	cold starts +736%
gpupack	binpack	energy kwh -2%; idle gpu hours powered -20%	wait mean min +16%; migrations 0 to 6.07
powersmooth	floor_safe	energy overhead pct -9%; throughput loss pct -44%; swing 1s mw -6%	max ramp mw s +127%
health	detect	goodput pct +10%; lost gpu hours -20%; restarts -29%; straggler node hours -99%	-
cooling	reset	pue -2%; cooling mwh -13%; inlet violation min -73%	-
inference	keda	tfft p95 s -18%; slo breach pct -58%; gpu hours -19%; preemptions -56%	-
containment	static	rogue overspend usd -96%; time to contain min -99%; peak subagents -59%	false stops +357%; honest work pct -7%
vmenergy	ratio	attr error pct -37%; worst vm error pct -34%	-

9. Threats to validity

- Competitors are emulated from documentation on a shared plant; their production binaries may behave differently, and their full products include features (spot pricing, instance-type selection, rebalancing) that this plant does not model.

- Simulated plants are fluid models: pods pack perfectly, boot delay is fixed, power is idle + dynamic x utilisation. Real clusters fragment and boot times vary.
- Live runs use kind: nodes are containers on one CI machine; power is modelled, not metered; the two arms ran on two different CI machines. Six 20-minute runs are evidence of behaviour, not of production savings.
- Development and held-out scenarios come from the same generator; held-out protects against tuning to seeds, not against a generator that differs from real traffic. Real traces (PlanetLab) are covered in the main benchmark report.
- The loss rule uses a 0.5% tolerance and a 95% interval on 30 scenarios; small true differences can be missed.

10. Reproduce

```

pip install -r requirements.txt && python verify.py
python tuning/b_league.py dev                # B vs A, choose settings (development seeds)
python tuning/b_league.py heldout tuning/B_SETTINGS_FROZEN.json # B vs A, held-out
python tuning/league.py heldout              # C vs all platforms
python tuning/bound.py                      # perfect-foresight frontier
python tools/mechanism.py                   # engine modes and compute
for m in rightsize coldstart gpupack powersmooth health cooling inference containment vmenergy; do python -m omnilab.bench $m heldout; done
GitHub Actions workflow 'benchmark' (commit message tag [bench]): live A vs B on kind
python tools/abc_report.py && python pilot/bench_pdf.py docs/OMNICOmpASS_ABC_REPORT.md docs/OMNICOmpASS_ABC_REPORT.pdf

```